

AUTHORED BY : ARPIT RAJ , LNMIIT JAIPUR , JULY 28, 2026

Path with Maximum Gold

asked in goldman sachs , geiko

Problem Understanding

Each cell contains

0 -> Empty cell (cannot enter)

>0 -> Gold present

Rules:

- Aadya can start from any gold cell.
- She can move in 4 directions.
- She cannot visit the same cell twice.
- Collect as much gold as possible.
- Return the maximum gold collectible.

DFS Logic & Example

our DFS should return

Maximum gold obtainable starting from this cell.

Suppose current cell contains **20**

Explore

Up	Down	Left	Right
----	------	------	-------

Suppose they return

5

18

7

11

Then

$$\text{Answer} = 20 + \max(5, 18, 7, 11) = 38$$

Base



Seen



Explore



Backtrack

C++ Implementation

```
class Solution {
public:
    int n;
    int m;
    vector<pair<int,int>> dir={{-1,0},{0,-1},{0,1},{1,0}};

    int dfs(int r, int c, vector<vector<int>>& grid){

        if(r<0 || c<0 || r>=n || c>=m){
            return 0;
        }

        if(grid[r][c]==0) return 0;
```

```

        //SEB start

        int temp=grid[r][c];
        grid[r][c]=0;
        int maxu=0;
        for(auto &it:dir){
            int nr=r+it.first;
            int nc=c+it.second;
            maxu=max(maxu,dfs(nr,nc,grid));
            //we will see kis direction se max goldd mil rha
        }

        grid[r][c]=temp;
        return grid[r][c]+maxu; //current cell + best direction loot
    }

    int getMaximumGold(vector<vector<int>>& grid) {

        //basic idea ye rhega ki
        //har cell se dfs start karo
        //maintain a maxans , to see which start cell gives max answer
        //while doing dfs we only take gold from the best direction

        n=grid.size();
        m=grid[0].size();
        int ans=0;
        for(int i=0; i<n; i++){
            for(int j=0; j<m; j++){

                if(grid[i][j]==0) continue;
                ans=max(ans,dfs(i,j,grid));
            }
        }

        return ans;
    }
};

```